

Amazon S3, carte mentale

PIÈGE classique à l'examen **À JOUR** diffère des notes du cours Sigles soulignés en pointillé : voir le glossaire en bas

S3

Simple Storage Service

01 Fondamentaux

- Stockage **objet** (pas bloc, pas fichier)
- Durabilité **11 9s** (99,999999999 %)
- Événements (create, delete, restore, réplication)
 - Cibles : Lambda, SNS, SQS
- ♦ **PIÈGE** Pas de scripting serveur (PHP, JSP, ASP.NET)
- Cas d'usage
 - Stockage et distribution de contenu (+ CloudFront)
 - Backup / archive → lifecycle vers Glacier
 - Data lake, big data
 - DR avec **CRR** (réplication asynchrone entre régions)
 - Site web statique

02 Buckets

- Nom **globalement unique** (tous comptes, toutes régions)
 - 3-63 caractères
 - minuscules, chiffres, tirets
 - pas de majuscules ni d'underscore
- Bucket **régional** : choix selon latence, coût, réglementation
- ♦ **PIÈGE** Nom et région **non modifiables** après création
- URL virtual-host-style
 - bucket.s3.<region>.amazonaws.com/<key>
- Pas de vrais dossiers : des **préfixes** dans la clé
 - 2021/math/John.txt → filtrer par préfixe
- Unité de facturation et de contrôle d'accès

03 Objets et metadata

- Objet = **data + metadata**, identifié par une **clé** unique
- Metadata système
 - Gérées par S3 : date de création, taille, version
 - Modifiables : storage class, chiffrement
- Metadata utilisateur
 - Préfixe obligatoire **x-amz-meta-**
 - Stockées avec l'objet, jamais traitées par S3
- Headers utiles à l'upload : ContentType, CacheControl (max-age=0 = pas de cache navigateur)

04 Opérations

- **PUT** : écrit toujours l'objet entier
 - ♦ Pas de mise à jour partielle
 - Upload simple : jusqu'à 5 GB
 - Multipart recommandé dès 100 MB
 - Multipart obligatoire > 5 GB, max 5 TB
 - Parts en parallèle, reprise possible
- **GET** : objet complet ou **byte-range** (connexion lente, traitement partiel)
- **DELETE** : un ou plusieurs objets (voir versioning)
- SDK Python : boto3.client("s3").upload_file(...)

05 Versioning

- Plusieurs versions d'une même clé, chacune avec un **version ID**
- Protège des écrasements, suppressions, bugs applicatifs
- ♦ **PIÈGE** Désactivé par défaut ; une fois activé, seulement **suspendu**, jamais désactivé
- Chaque version est facturée
- DELETE sans version ID
 - → delete marker devient la version courante → GET renvoie 404
- DELETE avec clé + version ID
 - → suppression **définitive** de cette version
- Requis pour **Object Lock** et pour la réplication

06 Chiffrement qui tient la clé, et qui peut lire

LES SIGLES

- **SSE** = *Server-Side Encryption* : c'est S3 qui chiffre et déchiffre
- **SSE-S3** : clé gérée par S3
- **SSE-KMS** : clé gérée par **KMS** (*Key Management Service*)
- **SSE-C** : clé fournie par le client (*Customer-provided key*)
- **Client-side (CSE)** : *Client-Side Encryption*, c'est toi qui chiffres avant l'upload
- **AES-256** : l'algorithme utilisé (*Advanced Encryption Standard*, clé de 256 bits)

LE PRINCIPE : 2 CLÉS (ENVELOPE ENCRYPTION)

Clé maître verrouille Clé de données verrouille Ton fichier

- **Clé de données** : une par objet, elle chiffre le fichier
- **Clé maître** : elle chiffre la clé de données
- S3 stocke le fichier chiffré + sa clé de données chiffrée
- Intérêt : on protège et contrôle **une seule** clé maître au lieu de millions

LES 4 MODES = QUI TIENT LA CLÉ MAÎTRE

MODE	QUI TIENT LA CLÉ	TOI, TU FAIS	QUI PEUT LIRE
SSE-S3	S3, invisible	Rien	Tous ceux qui ont les droits S3
SSE-KMS	KMS, dans ton compte	Nommer la clé à l'upload	Droits S3 et kms:Decrypt , chaque lecture loggée
SSE-C	Toi, hors AWS	Envoyer la clé à l'upload et au download	Ceux qui ont ta clé
Client-side	Toi	Chiffrer toi-même avant l'upload	Ceux qui ont ta clé ; AWS ne voit rien

Dans les 3 modes SSE, S3 fait le travail. Seule l'origine de la clé change.

TU CHIFFRES DÉJÀ SANS LE SAVOIR

- **À JOUR** Depuis janv. 2023, **SSE-S3 automatique** sur tout nouvel objet
- **Transparent** : S3 chiffre à l'écriture, déchiffre à la lecture
- Vérifier : aws s3api head-object --bucket b --key k
 - "ServerSideEncryption": "AES256" = SSE-S3 ; "aws:kms" = SSE-KMS
- ♦ **PIÈGE** SSE-S3 protège contre le **vol de disque**, pas contre quel'un qui a les droits IAM
- Donc au quotidien, ta vraie protection reste les **permissions** (branche 07)
- SSE-S3 : AES-256, rotation de la clé maître automatique par S3

POURQUOI ALLER PLUS LOIN QUE SSE-S3

- **Deuxième serrure** : un dev avec **s3:*** voit les fichiers mais ne peut pas les ouvrir sans la clé KMS
- **Coupure d'urgence** : fuite de credentials → désactiver la clé KMS, plus personne ne lit
- **Audit** : CloudTrail loggue chaque usage de la clé (qui, quand, quel objet)
- **Contrôle de la clé** : tu choisis qui l'utilise (key policy) et tu gères sa rotation
- **Obligation légale** (banque, santé) : la clé ne doit pas être chez AWS → SSE-C ou client-side

SSE-KMS EN PRATIQUE

- ♦ **PIÈGE** Lire demande s3:GetObject et kms:Decrypt, sinon AccessDenied
- Chaque GET/PUT appelle KMS, qui a des **quotas** → throttling possible
 - Solution : **S3 Bucket Key** (beaucoup moins d'appels KMS)

```
s3.upload_file("f.pdf", "bucket", "f.pdf", ExtraArgs={
    "ServerSideEncryption": "aws:kms",
    "SSEKMSKeyId": "arn:aws:kms:...:key/abcd"})
# download : rien à préciser, S3 appelle KMS pour toi
```

SSE-C EN PRATIQUE

- Clé envoyée à **chaque requête**, S3 l'oublie aussitôt
- HTTPS obligatoire
- ♦ Clé perdue = données perdues

```
args = {"SSECustomerAlgorithm": "AES256",
        "SSECustomerKey": ma_cle}
s3.upload_file(..., ExtraArgs=args)
s3.download_file(..., ExtraArgs=args) # même clé
```

CLIENT-SIDE EN PRATIQUE

- S3 ne stocke que du texte chiffré, il ne sait même pas que c'est chiffré
- Pourquoi : AWS ne doit jamais voir le clair (loi, contrat) ; défense en profondeur (bucket public par erreur, credentials volés → toujours illisible) ; chiffrement de bout en bout depuis la machine source
- Coût : gestion des clés à ta charge, Athena / S3 Select / Lambda ne lisent plus le contenu, plus de code
- ♦ Clé perdue = données perdues
- Outils : **Amazon S3 Encryption Client**, **AWS Encryption SDK** (jamais son propre algo)
- La clé maître peut rester dans KMS, le chiffrement se fait quand même en local

```
chiffre = mon_chiffrement(open("f.pdf", "rb").read(), ma_cle)
s3.put_object(Bucket="bucket", Key="f.pdf", Body=chiffre)
```

EN TRANSIT ET RÈGLES IMPOSÉES

- **HTTPS / TLS**
 - **À JOUR** TLS 1.2 minimum
- Forcer HTTPS : Deny si aws:SecureTransport = false
- Forcer un chiffrement : Deny si le header **x-amz-server-side-encryption** ≠ AES256 / aws:kms
- Changer le chiffrement par défaut ne rechiffre pas les objets existants

LEQUEL CHOISIR

- Site statique, assets publics → **SSE-S3** (rien à faire)
- Données sensibles, audit, révocation → **SSE-KMS**
- Clé hors d'AWS, mais AWS chiffre → **SSE-C**
- AWS ne doit rien voir en clair → **client-side**

07 Contrôle d'accès qui peut faire quoi, et qui gagne en cas de conflit

LES 3 TYPES ET QUAND LES UTILISER

TYPE	ATTACHÉ À	UTILISER QUAND
Policy IAM identity-based	User, groupe, rôle	Une identité de ton compte accède au bucket (ex. rôle d'une Lambda). Cas le plus courant.
Bucket policy resource-based	Le bucket (JSON)	Accès public , accès cross-account , ou règle pour tous (HTTPS, chiffrement, VPC endpoint, IP)
ACL resource-based	Bucket ou objet (XML)	Ne plus utiliser. Théorie d'examen seulement.

Ton code dans ton compte → IAM. Public, cross-account ou règle globale → bucket policy. ACL → oublier.

- ACL, pour la théorie : c'était le seul moyen de gérer un objet uploadé par un **autre compte** (le bucket owner ne le possédait pas). Régulé par *Bucket owner enforced*.
- Bucket policy = même langage JSON qu'IAM, avec en plus un champ **Principal** (qui)

QUI GAGNE : L'ORDRE D'ÉVALUATION

- Les policies ne s'annulent pas, AWS les évalue **toutes ensemble**
- **1. Refus implicite** : aucun Allow nulle part → refusé
- **2. Un seul Allow suffit** (même compte) : IAM *ou* bucket policy
- ♦ **3. Un Deny explicite gagne toujours**, même contre 10 Allow
- Exemple : rôle IAM avec **s3:*** + bucket policy qui refuse le HTTP → requête HTTP refusée

CROSS-ACCOUNT

- ♦ **PIÈGE** Un seul Allow ne suffit plus : il en faut **des deux côtés**
 - Bucket policy du compte A autorise le compte B
 - Policy IAM de l'identité du compte B autorise l'action

AU-DESSUS DE TOUT

- Tout est **privé par défaut**
- **Block Public Access** (bucket ou compte) : bloque une bucket policy "Principal": "*" même valide
- **À JOUR** **Object Ownership** = *Bucket owner enforced* par défaut depuis avril 2023 : ACL désactivées, le bucket possède tous les objets

- Pour SSE-KMS, la **key policy** KMS s'ajoute : il faut aussi **kms:Decrypt**

PRESIGNED URL

- GET ou PUT sur **un objet**, sans donner d'autre droit
- Pas un 4e type : elle **emprunte les permissions** de celui qui la signe (sans droit → 403)
- Max **7 jours** (credentials d'IAM user)
- ♦ Signée par un rôle / STS : expire au plus tard avec la session

CORS

- Du JS chargé depuis un autre domaine appelle le bucket → le navigateur exige CORS
- Config JSON : AllowedOrigins, AllowedMethods, AllowedHeaders, MaxAgeSeconds
- Max **100** règles

EXEMPLES CONCRETS

1. Policy IAM sur le rôle d'une Lambda (ton compte)

```
{ "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::mon-bucket/*" }
```

2. Bucket policy : lecture publique (site statique)

```
{ "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::mon-bucket/*" }
```

3. Bucket policy : cross-account (+ Allow IAM côté compte B)

```
{ "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::222233334444:root" },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::mon-bucket/*" }
```

EXEMPLES : IMPOSER UNE RÈGLE AVEC DENY

4. Refuser tout accès sans HTTPS

```
{ "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::mon-bucket",
               "arn:aws:s3:::mon-bucket/*"],
  "Condition": { "Bool": { "aws:SecureTransport": "false" } } }
```

5. Refuser un upload qui n'est pas en SSE-KMS

```
{ "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::mon-bucket/*",
  "Condition": { "StringNotEquals": { "s3:x-amz-server-side-encryption": "aws:kms" } } }
```

Resource : **mon-bucket** = le bucket lui-même (ListBucket) ; **mon-bucket/*** = les objets (Get/Put/Delete).

08 Site web statique

- **S3 website hosting** seul
 - bucket.s3-website-<region>.amazonaws.com
 - Bucket public + bucket policy
 - index.html / error.html, redirections
 - ♦ HTTP seulement
- **S3 + CloudFront** (prod)
 - Bucket **privé**, accès via **OAC** (remplace OAI)
 - HTTPS + domaine custom
 - ♦ Certificat ACM en us-east-1
- **Amplify Hosting** : S3 + CloudFront managés + CI/CD Git
- Déployer : aws s3 sync ./build s3://bucket --delete puis invalidation CloudFront

09 Glossaire des sigles

S3	Simple Storage Service	IAM	Identity and Access Management (users, rôles, policies)	ARN	Amazon Resource Name (identifiant unique d'une ressource)
ACL	Access Control List	SSE	Server-Side Encryption (S3 chiffre)	SSE-S3	SSE avec clé gérée par S3
SSE-KMS	SSE avec clé gérée par KMS	SSE-C	SSE avec Customer-provided key	CSE	Client-Side Encryption (tu chiffres)
KMS	Key Management Service	AES-256	Advanced Encryption Standard, clé 256 bits	TLS	Transport Layer Security (le S de HTTPS)
STS	Security Token Service (credentials temporaires)	CORS	Cross-Origin Resource Sharing	CRR	Cross-Region Replication
DR	Disaster Recovery	OAC	Origin Access Control (CloudFront → bucket privé)	OAI	Origin Access Identity (ancien OAC)
ACM	AWS Certificate Manager	SNS	Simple Notification Service	SQS	Simple Queue Service
SDK	Software Development Kit (boto3 en Python)	VPC	Virtual Private Cloud		

Les réflexes pour l'examen

1. Fichier > 5 GB → multipart obligatoire. Dès 100 MB → multipart recommandé.
2. Modifier un bout d'objet → impossible, on réécrit l'objet entier.
3. Supprimé par erreur avec versioning → retirer le delete marker.
4. Versioning ne se désactive pas, il se suspend.
5. Deny explicite quelque part → toujours refusé.
6. Autre compte AWS doit accéder → bucket policy + Allow IAM côté B.
7. Bucket policy publique sans effet → Block Public Access activé.
8. Accès temporaire à un objet sans créer d'user IAM → presigned URL.
9. Presigned URL qui expire trop tôt → signée avec des credentials temporaires.
10. Audit de qui a utilisé la clé → SSE-KMS.
11. AccessDenied sur un objet SSE-KMS malgré les droits S3 → manque kms:Decrypt.
12. Throttling KMS sur S3 → S3 Bucket Key.
13. Clé hors d'AWS mais chiffrement par AWS → SSE-C.
14. AWS ne doit jamais voir les données en clair → client-side.
15. Erreur CORS dans le navigateur → configurer CORS sur le bucket.
16. Site en HTTPS → CloudFront, pas le website endpoint.
17. Déclencher un traitement à l'upload → S3 event → Lambda / SQS / SNS.
18. Ressource arn:aws:s3:::bucket ≠ bucket/* : ListBucket vise le bucket, GetObject vise les objets.
19. SSE-S3 ne protège pas contre un accès IAM trop large : ce sont les permissions qui protègent.